Formation Kubernetes

Session 3 — Ingress et Traefik

Reflekt Lab | 22h de formation | 11 sessions

Agenda de la session

Le problème de l'exposition

Pourquoi NodePort et LoadBalancer ne suffisent pas à l'échelle

10 min

Ingress Controller

Le concept, les options, pourquoi Traefik

15 min

Middlewares et sécurité

Rate-limiting, headers, TLS, cert-manager

10 min

Démo live

Le module traefik/ de l'infra RFLKT — Helm + Terraform

10 min

Travaux pratiques

Routing multi-services avec Traefik et IngressRoutes

1h10

Recap — Session 2

Deployment

Gère le cycle de vie des pods, rolling updates, rollbacks

Service

Point d'accès stable — ClusterIP, NodePort, LoadBalancer

DNS interne

service.namespace.svc.cluster.local — résolution automatique

ConfigMap

Configuration externalisée, montée en volume ou en variable d'env

Question : comment accéder à nos services depuis l'extérieur du cluster ?

Le problème : exposer des services

NodePort — Approche naïve

```
api.example.com:30080
web.example.com:30081
admin.example.com:30082
```

Ports non-standard (30000-32767)
Pas de TLS natif
Pas de routing par domaine
1 port = 1 service

LoadBalancer — Cher

```
1 IP externe par service
~18 $/mois chacun (GCP)
```

10 services = 10 LoadBalancers
= 10 IPs publiques
= ~180 $/mois juste en LB
Pas de routing intelligent

Il nous faut un point d'entrée unique et intelligent

Ingress Controller — La solution

→

Ingress Controller

(Traefik)

1 seul LoadBalancer
1 seule IP publique
Routing par Host / Path
TLS termination
Middlewares (rate-limit...)

→

Service:api

Service:web

Service:admin

Un seul point d'entrée, routing intelligent par règles déclaratives

Les Ingress Controllers

	Traefik	Nginx Ingress	Istio Gateway
CRDs natifs	✅ IngressRoute	❌ Annotations	✅ VirtualService
Dashboard	✅ Intégré	❌ Externe	✅ Kiali
Auto-discovery	✅ Watch API K8s	✅ Watch API K8s	✅ Watch API K8s
TLS auto	✅ Let's Encrypt	✅ cert-manager	✅ cert-manager
Middlewares	✅ Rate-limit, auth...	⚠ Via annotations	✅ Policies
Complexité	✅ Simple	✅ Simple	⚠ Service mesh

Nous utilisons Traefik v3 : CRDs puissants, simple à déployer, dashboard intégré

IngressRoute — Anatomie d'une route

```yaml
apiVersion: traefik.io/v1alpha1
kind: IngressRoute
metadata:
  name: api-ingressroute
  namespace: exercices
spec:
  entryPoints:
    - web
  routes:
    - match: Host(`api.training.local`)
      kind: Rule
      middlewares:
        - name: ratelimit
      services:
        - name: api
          port: 80
```

CRD Traefik

Pas un Ingress K8s standard

entryPoints

web (80) ou websecure (443)

match

Règle : Host, Path, Headers...

middlewares

Pipeline de traitement

services

Cible : Service K8s + port

Middlewares — Le pipeline de traitement

Requête entrante → **Rate Limit** (100 req/s max) → **Headers** (CORS, HSTS) → **Auth** (Basic / JWT) → **StripPrefix** (/api → /) →

Rate Limiting

```yaml
spec:
  rateLimit:
    average: 100
    burst: 200
```

Protège contre les abus et le DDoS

Security Headers

```yaml
spec:
  headers:
    customResponseHeaders:
      X-Frame-Options: DENY
      X-Content-Type-Options: nosniff
```

En-têtes HTTP de sécurité standard

StripPrefix

```yaml
spec:
  stripPrefix:
    prefixes:
      - /api
```

Retire un préfixe avant de transmettre

TLS / SSL — Certificats

Traefik + LE

- Gestion automatique
- Challenge HTTP-01 / DNS-01
- Renouvellement 90j auto

⚠ LE uniquement

cert-manager

- Multiple issuers
- LE, Vault, self-signed
- Standard de l'industrie

Opérateur à installer

Cloud Managed

- GCP / AWS / Azure certs
- LB porte le certificat
- Zéro maintenance

⚠ LB supplémentaire

Bonnes pratiques TLS en production

TLS 1.3 minimum
— TLS 1.0/1.1 désactivés, TLS 1.2 toléré si clients legacy

Cipher suites modernes
— ECDHE + AES-GCM, pas de CBC, pas de RSA key exchange

HSTS + Redirect HTTP → HTTPS
— Strict-Transport-Security: max-age=31536000; includeSubDomains

Chez RFLKT : GCP Managed Certs + Cloudflare (TLS termination en edge) → Traefik reçoit du HTTP interne

Pièges et réalités production

Single Point of Failure

Si Traefik tombe, tout le trafic est coupé. Pas de failover automatique.

Solution :

replicas: 2+, PodDisruptionBudget, monitoring/alerting dédié, health checks LB

Let's Encrypt + Recreate Strategy

LE stocke les certs sur un volume. Recreate = volume détaché → challenge LE re-exécuté → downtime si rate-limited.

Solution :

Utiliser RollingUpdate, ou stocker les certs dans un Secret K8s / provider externe

Liveness Probes sur Traefik

Sans probes, un pod Traefik crashé reste dans le Service. Le LB envoie du trafic vers un pod mort.

Solution :

livenessProbe: /ping, readinessProbe: /ping, failureThreshold adapté

Cipher Suites & TLS 1.3

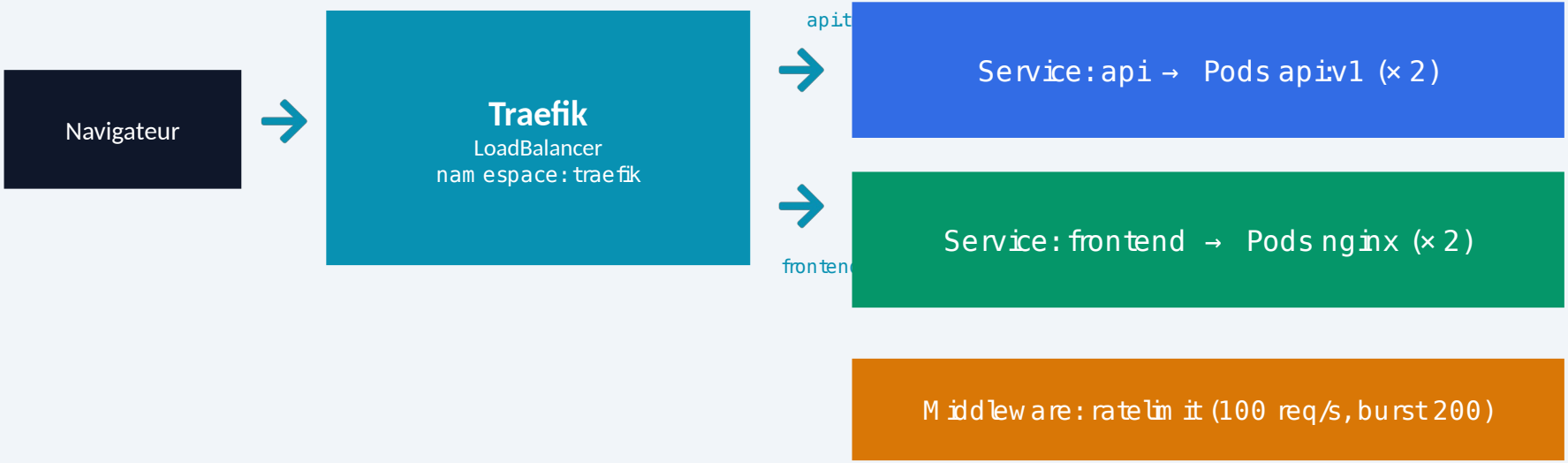
Config par défaut = TLS 1.2 avec ciphers legacy. Vulnérable aux attaques downgrade (POODLE, BEAST).

Solution :

minVersion: VersionTLS13, cipherSuites explicites dans la config Traefik

"En production, la question n'est pas SI ça va casser, mais QUAND"

Architecture du TP



Étapes du TP

1. Installer Traefik via Helm
2. IngressRoute API
3. Tester avec curl
4. IngressRoute Frontend
5. Rate-limiting middleware

Durée : 1h10 | Repo : k8s-training/labs/session-03-ingress/

Démo live — Traefik en production

Infrastructure RFLKT — Ce que vous construirez aux sessions 7-9

modules/traefik/

Helm chart déployé via Terraform — values.yaml avec config TLS, dashboard, entrypoints

modules/application/

Chaque app a son IngressRoute auto-générée — routage par domaine avec middleware chain

modules/gcp_managed_certs/

Certificats SSL managés par GCP — renouvellement automatique, zéro maintenance

modules/cloudflare_dns/

DNS automatique — quand on ajoute une app, le record DNS se crée tout seul

"En production, tout est automatisé — de l'ajout d'un service au certificat SSL"

Récap & Mini-défi

Ingress Controller = point d'entrée unique vers le cluster

IngressRoute = règle déclarative (Host, Path, middlewares)

Middlewares = rate-limit, headers, auth — pipeline configurable

TLS = indispensable en prod (cert-manager ou Traefik built-in)

Mini-défi pour la prochaine session

1. Créez un middleware basicAuth pour protéger l'accès à un service
2. Créez un middleware headers pour ajouter les en-têtes de sécurité (CORS, HSTS)
3. Chaînez plusieurs middlewares sur une même IngressRoute